

Call for Expressions of Interest

<i>Date of Issue:</i>	September 2026
<i>Expected Completion Date:</i>	May 2027
<i>Project Title:</i>	Simple logic proof tutor/practice tool
<i>Company:</i>	University of Nottingham

Contact Details

<i>Lead Contact:</i>	Jason Atkin
<i>Lead E-mail:</i>	Jason.atkin@nottingham.ac.uk
<i>Additional Contact:</i>	Additional contact name
<i>E-mail:</i>	Additional contact e-mail
<i>Additional Contact:</i>	Additional contact name
<i>E-mail:</i>	Additional contact e-mail
<i>Additional Contact:</i>	Additional contact name
<i>E-mail:</i>	Additional contact e-mail

Additional Info

<i>Students must sign NDA:</i>	☐
<i>Project uses personal data:</i>	☐
<i>Custom hardware required:</i>	☐ If Applicable: Please provide details
<i>Relevant research areas (if applicable)¹:</i>	Java/OO programming, maths

Project Background Information

Any of you who did your first year at UoN UK last year will know that I taught you propositional and predicate logic in the first semester. You will also know that I taught OO/Java programming in semester 2. This project brings these two topics together to use Java/OO programming to develop something that will help the students in future years to better understand propositional logic, predicate logic and the types of simple proofs that we do in these in the first semester (which, admittedly, will not feel simple to students at the time).

Some students struggle to understand truth tables and how to split up a proposition into its atomic parts. This project should help them by allowing them to experiment with breaking it up, and either ask students to specify truth values and tell them the validity or not, or provide truth table columns, depending upon mode of operation.

Many students struggle to understand how to do a step-by-step annotated proof, including raising simple flags for at least implication and contradiction but ideally also let flags. Flags within flags will be key to understand as often causes students difficulties. As is restricting things within a flag to not be visible outside. This project should help them by allowing them to specify premises and conclusion(s) and build up the next line in a proof, or raise a

¹ An overview of different research areas in the school of computer science can be found [here](#).

flag, incrementally, so that students can easily try things. The aim is that allowing students to say what they want, and having the tool do it, will split the task for them, allowing them to understand the logic of building a proof better without worrying so much about the format. E.g. it would offer options to apply for the next step then do it visually so that students can see it.

Ideally also support predicate proofs – with let flags/assumptions and pick – so students can see how to get from the predicate premises into a propositional sub-proof on specific cases, and then back to the predicate conclusion.

Requirements

The group must provide:

A program in Java which will run as a stand-alone executable under a java virtual machine, which will display a user interface to users, allowing them to experiment with different propositions, predicated, logical operators, and proofs, as seen in the semester 1 maths module.

A Visualisation of truth tables for propositions – including breaking it up into atomic steps and allowing users to enter results for checking, or to get the right answer, as well as to compare columns, e.g. to visually see what equivalence or entailment (or not) mean.

A visualisation of propositional and ideally also predicate proofs, allowing different equivalence, entailment or assumption flag raising operations to be tried by users, so that they can experiment with proofs and see that they are not as difficult as they thought. Ease of use is key here, to make this simple.

A 'test me' mode would be good, where students could use it to test their knowledge – as that is a good way to learn. And the ability to provide some test propositions/predicated for premises and conclusions for this purpose. E.g. from an input file which could provide proofs of various difficulties. (This could also optionally include suggested solutions.) There are plenty of examples in the course which could be used to start this.

You will need at least these elements:

- Ability to enter a compound proposition, parse it into a data structure (I suggest a tree or operators and operands), and to go back to a readable proposition from a data structure.
- Ability to display a truth table for a proposition – with component parts (e.g. a row for each node of the tree) to help students understand truth tables and fundamental logical operators.
- Clear visualisation of an annotated proof, including multiple flags.
- Ability to apply entailment or equivalency operators to your data structure, to get another proposition. (This is needed for the proofs.)
- Ability to handle flag raising, such as for contradiction, implication, let. And allow pick to effectively switch from predicate to propositional sub-proofs.
- It must be easy to maintain, if features, equivalencies or entailments need to be added later. (Good OO design should allow this.)

Skills	
<i>Highly desirable:</i> <i>1st year programming paradigms skills – designing and creating programs in Java.</i> <i>1st year maths skills</i>	<i>Desirable:</i> User interface design skills, including ease of use. Ability to explain concepts to others and understand their difficulties – to understand how to best design a program to help clarify these concepts to other students. Keenness to help next year's students understand that these concepts are not as difficult as they seem. Understanding of recursion. (Will ease some things, like the parsing or flags in flags.) Innovation – there is lots of flexibility about how things are done, or how to help students learn these things.
Submission / Award information	
<i>Date of Submission:</i>	September 2026
<i>Date of Pitch:</i>	October 2026
<i>Notification of award:</i>	October 2026

For further information on this proposal, the team representative should in the first instance e-mail the lead contact stated above at the e-mail address provided.